In [3]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

In [4]:
```
data = pd.read_csv(r"C:\Users\ggulz\Downloads\country_wise_latest.csv")
data
```

Out[4]:

	Country/Region	Confirmed	Deaths	Recovered	Active	New cases	New deaths	New recovered	Deaths / 100 Cases	Recovered / 100 Cases	Deaths / 100 Recovered	Confirmed last week	1 week change	1 week % increase	WHO Region
0	Afghanistan	36263	1269	25198	9796	106	10	18	3.50	69.49	5.04	35526	737	2.07	Eastern Mediterranean
1	Albania	4880	144	2745	1991	117	6	63	2.95	56.25	5.25	4171	709	17.00	Europe
2	Algeria	27973	1163	18837	7973	616	8	749	4.16	67.34	6.17	23691	4282	18.07	Africa
3	Andorra	907	52	803	52	10	0	0	5.73	88.53	6.48	884	23	2.60	Europe
4	Angola	950	41	242	667	18	1	0	4.32	25.47	16.94	749	201	26.84	Africa
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
182	West Bank and Gaza	10621	78	3752	6791	152	2	0	0.73	35.33	2.08	8916	1705	19.12	Eastern Mediterranean
183	Western Sahara	10	1	8	1	0	0	0	10.00	80.00	12.50	10	0	0.00	Africa
184	Yemen	1691	483	833	375	10	4	36	28.56	49.26	57.98	1619	72	4.45	Eastern Mediterranean
185	Zambia	4552	140	2815	1597	71	1	465	3.08	61.84	4.97	3326	1226	36.86	Africa
186	Zimbabwe	2704	36	542	2126	192	2	24	1.33	20.04	6.64	1713	991	57.85	Africa

187 rows × 15 columns

Data Cleaning

In [5]:
```
data.shape
```

Out[5]:
```
(187, 15)
```

In [6]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 187 entries, 0 to 186
Data columns (total 15 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Country/Region          187 non-null    object
 1   Confirmed               187 non-null    int64
 2   Deaths                  187 non-null    int64
 3   Recovered               187 non-null    int64
 4   Active                  187 non-null    int64
 5   New cases               187 non-null    int64
 6   New deaths              187 non-null    int64
 7   New recovered           187 non-null    int64
 8   Deaths / 100 Cases      187 non-null    float64
 9   Recovered / 100 Cases   187 non-null    float64
 10  Deaths / 100 Recovered  187 non-null    float64
 11  Confirmed last week     187 non-null    int64
 12  1 week change           187 non-null    int64
 13  1 week % increase       187 non-null    float64
 14  WHO Region              187 non-null    object
dtypes: float64(4), int64(9), object(2)
memory usage: 22.0+ KB
```

In [7]:
```
data.isnull().sum()
```

Out[7]:
```
Country/Region            0
Confirmed                 0
Deaths                    0
Recovered                 0
Active                    0
New cases                 0
New deaths                0
New recovered             0
Deaths / 100 Cases        0
Recovered / 100 Cases     0
Deaths / 100 Recovered    0
Confirmed last week       0
1 week change             0
1 week % increase         0
WHO Region                0
dtype: int64
```

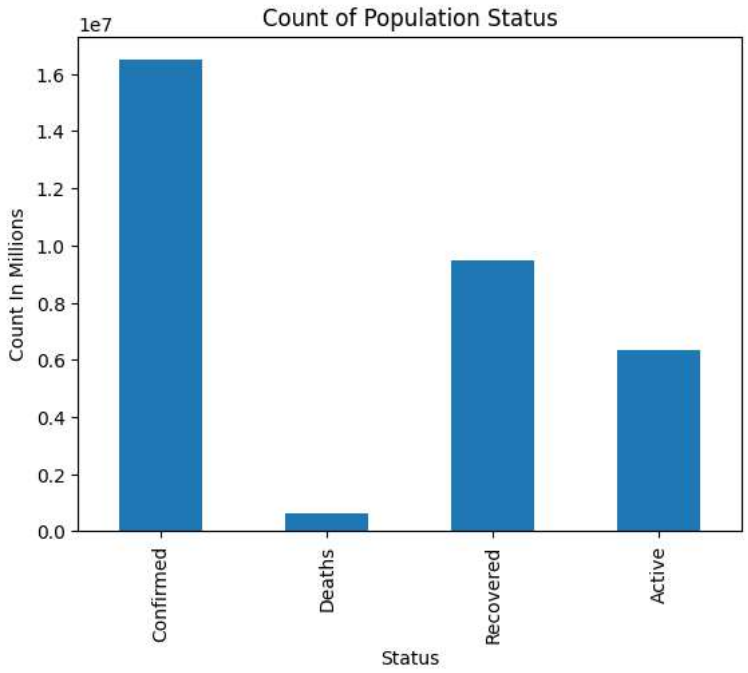
In [8]:
```
duplicate_rows_data = data[data.duplicated()]
print("number of duplicate rows: ", duplicate_rows_data.shape[0])
```

```
number of duplicate rows:  0
```

Data Analysis

In [9]:
```
status_count = ['Confirmed', 'Deaths', 'Recovered', 'Active']
column_sums = data[status_count].sum().plot(kind='bar')
plt.xlabel("Status")
plt.ylabel("Count In Millions")
plt.title("Count of Population Status")
```

Out[9]:
```
Text(0.5, 1.0, 'Count of Population Status')
```



In [10]:
```
Region = data.groupby(by = 'WHO Region').sum()
Region.head()
```

```
C:\Users\ggulz\AppData\Local\Temp\ipykernel_5112\1995356871.py:1: FutureWarning: The default value of numeric_only in DataFrameGroupBy.sum is deprecated. In a future version, numeric_on
ly will default to False. Either specify numeric_only or select only columns which should be valid for the function.
  Region = data.groupby(by = 'WHO Region').sum()
```

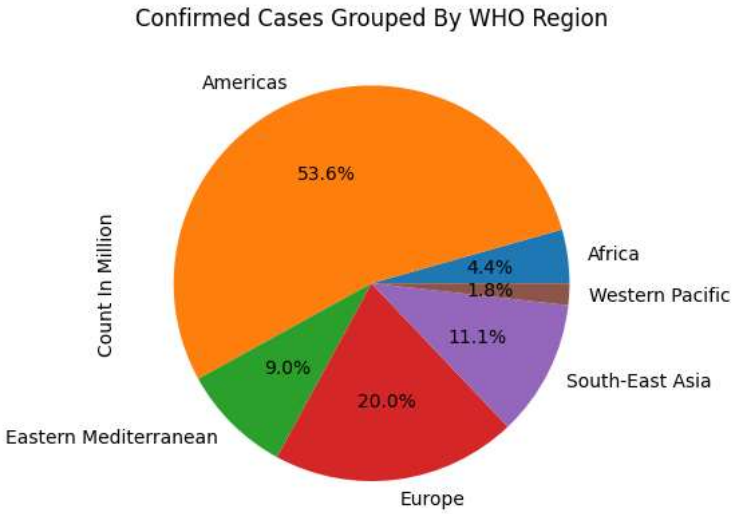
Out[10]:

WHO Region	Confirmed	Deaths	Recovered	Active	New cases	New deaths	New recovered	Deaths / 100 Cases	Recovered / 100 Cases	Deaths / 100 Recovered	Confirmed last week	1 week change	1 week % increase
Africa	723207	12223	440645	270339	12176	353	14563	110.71	2736.71	NaN	608120	115087	868.15
Americas	8839286	342732	4468616	4027938	129531	3555	94776	106.84	2180.20	NaN	7815198	1024088	571.59
Eastern Mediterranean	1490744	38339	1201400	251005	12410	445	14843	78.39	1465.05	NaN	1398847	91897	230.61
Europe	3299523	211144	1993723	1094656	22294	304	11732	235.11	3843.56	NaN	3146816	152707	435.10
South-East Asia	1835297	41349	1156933	637015	48993	734	37582	12.96	667.04	19.56	1478283	357014	85.13

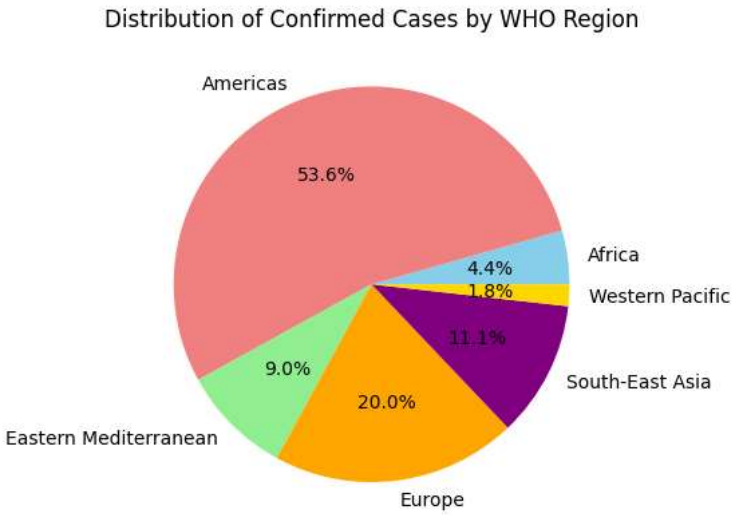
Confirmed Cases in Each Region

In [11]:
```
Region['Confirmed'].plot(kind='pie',autopct='%1.1f%%')
plt.title('Confirmed Cases Grouped By WHO Region')
plt.ylabel('Count In Million')
```

Out[11]:
```
Text(0, 0.5, 'Count In Million')
```

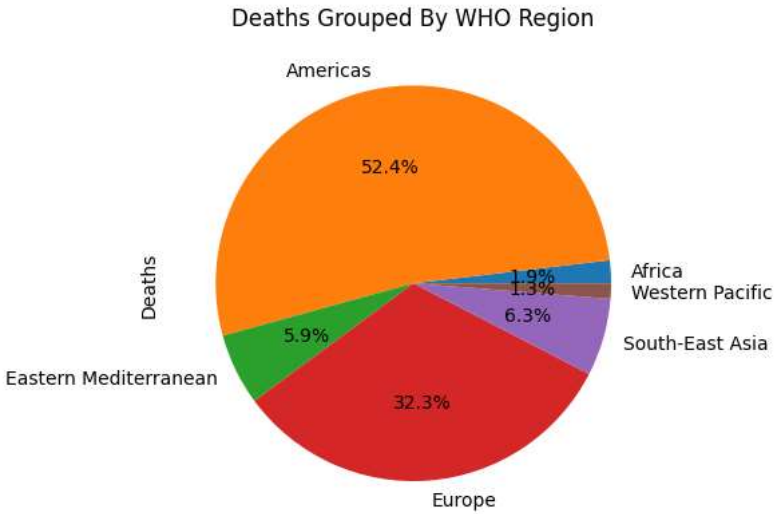


In [12]:
```
colors = ['skyblue', 'lightcoral', 'lightgreen', 'orange', 'purple', 'gold', 'pink']
Region['Confirmed'].plot(kind='pie', autopct='%1.1f%%', colors=colors)
plt.title('Distribution of Confirmed Cases by WHO Region')
plt.ylabel('')
plt.show()
```

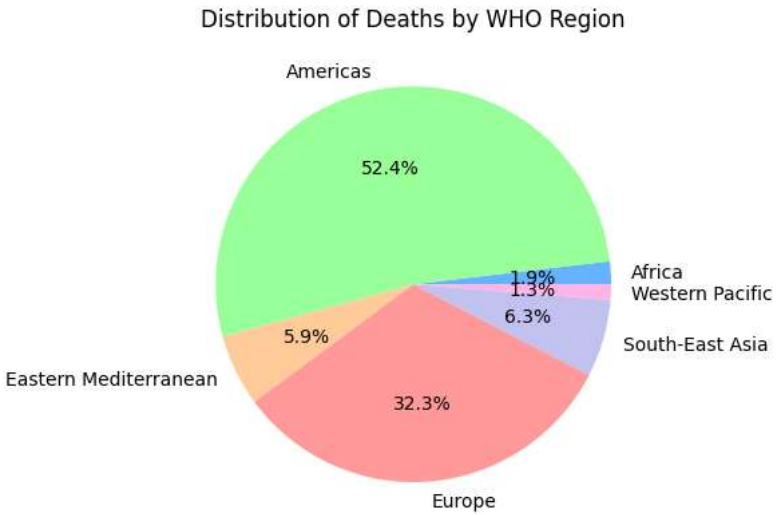


In [13]:
```
Region['Deaths'].plot(kind='pie',autopct='%1.1f%%')
plt.title('Deaths Grouped By WHO Region')
```

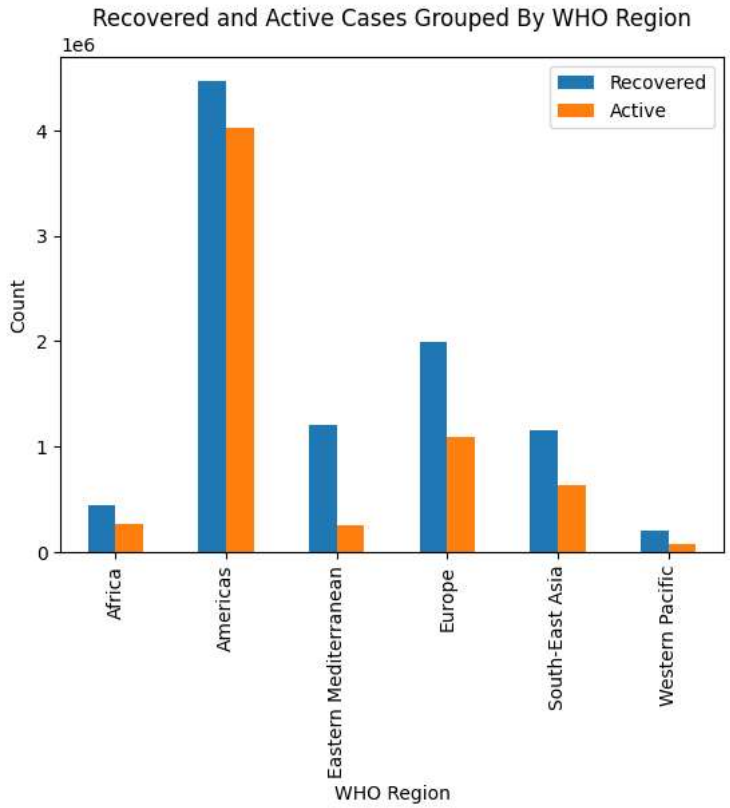
Out[13]: Text(0.5, 1.0, 'Deaths Grouped By WHO Region')



In [14]:
```
colors = ['#66b3ff', '#99ff99', '#ffcc99', '#ff9999', '#c2c2f0', '#ffb3e6', '#ff6666']
Region['Deaths'].plot(kind='pie', autopct='%1.1f%%', colors=colors)
plt.title('Distribution of Deaths by WHO Region')
plt.ylabel('')
plt.show()
```

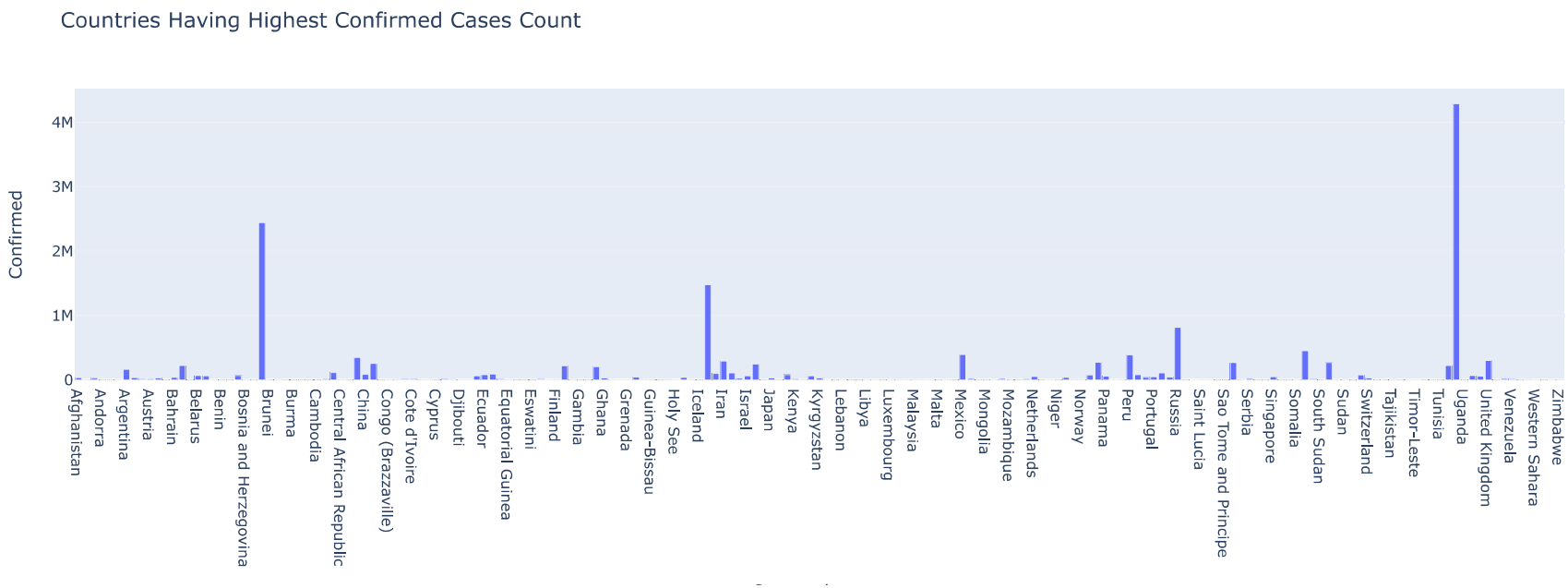


In [15]:
```
Region[['Recovered', 'Active']].plot(kind='bar')
plt.title('Recovered and Active Cases Grouped By WHO Region')
plt.ylabel('Count')
plt.show()
```

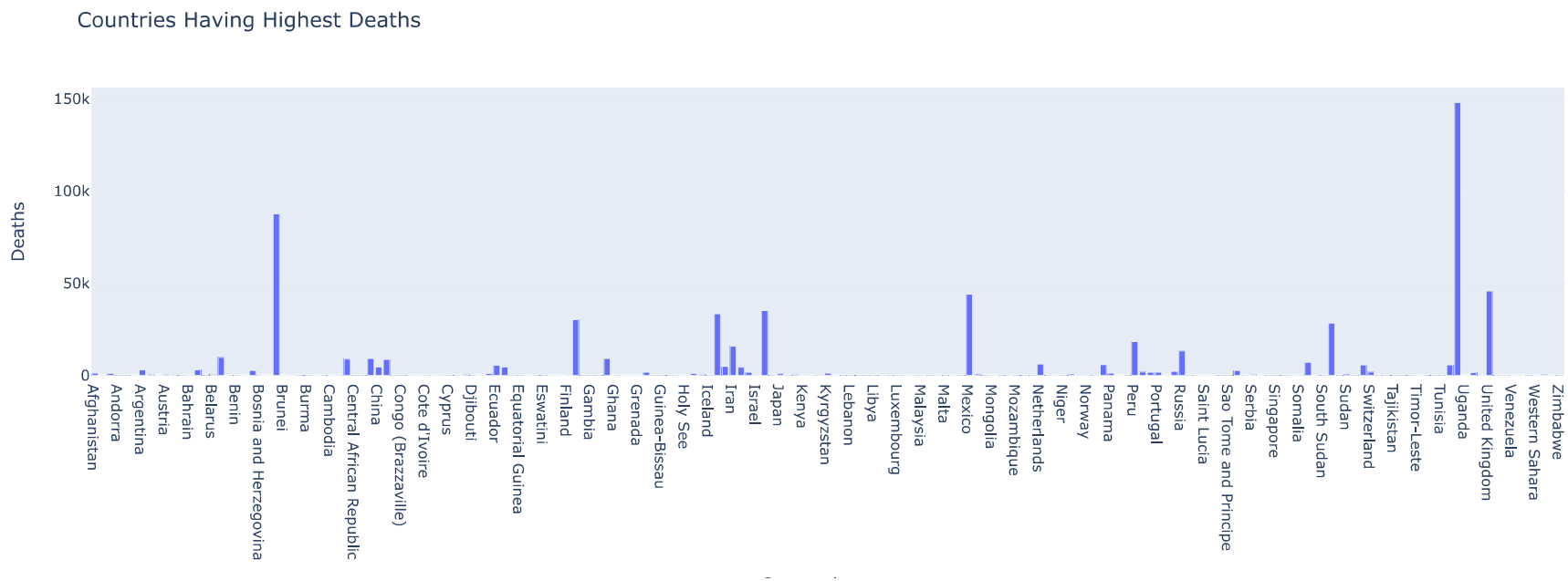


In [16]:
```
import plotly.express as px
```

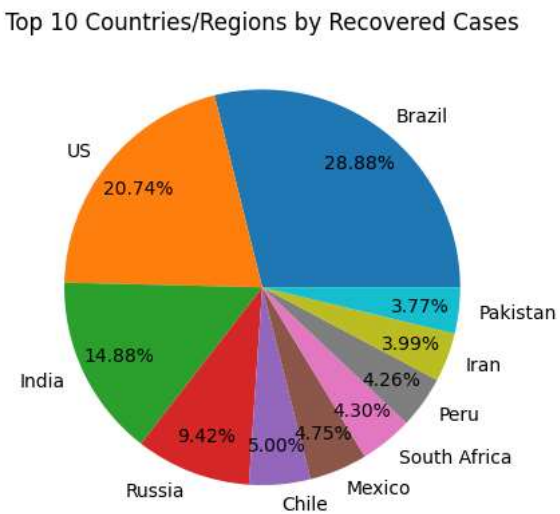
In [17]:
```
fig = px.bar(data, x="Country/Region", y="Confirmed", title="Countries Having Highest Confirmed Cases Count")
fig.show()
```



In [18]:
```
fig = px.bar(data, x="Country/Region", y="Deaths", title="Countries Having Highest Deaths")
fig.show()
```



In [19]:
```
top_recovered = data.sort_values(by='Recovered', ascending=False)[:10]
plt.pie(top_recovered['Recovered'], labels=top_recovered['Country/Region'], autopct="%0.2f%%", pctdistance=0.8)
plt.title('Top 10 Countries/Regions by Recovered Cases')
plt.show()
```



In [ ]: